AI-powered Virtual Humans

LLM-Powered Cognitive Architectures



Module 2 - Chapter 02
Building Stateful Agents with Memory

From a List to a Checkpointer [1] [3]

Module 1: your list

Append human and AI messages. Resend the whole list on each call.

LangGraph

Messages live in the graph state. A checkpointer saves them per conversation.

Same idea; the graph manages it for us.

Messages in State [2]

```
from typing import Annotated
from langgraph.graph.message import add_messages

class ChatState(TypedDict):
    messages: Annotated[list, add_messages]
```

Annotated attaches a reducer to the messages key.

Replace or Append [2]

Default (Chapter 01)

A new value replaces the old one.

add_messages

New messages are appended to the list.

MessagesState provides the same messages key and reducer, ready-made.

Open 01_memory.ipynb: 1. Messages in state

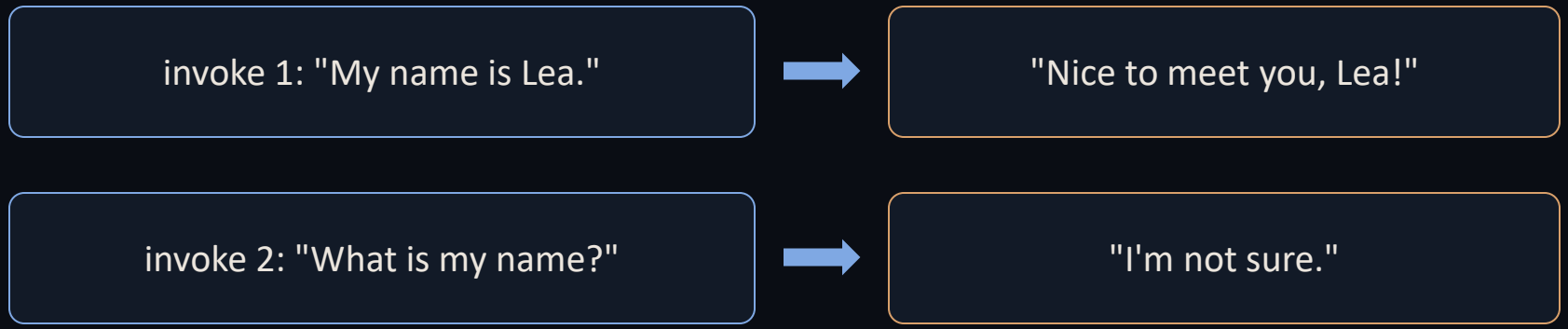
A Chatbot Node [2]

```
prompt = ChatPromptTemplate.from_messages([
    SystemMessagePromptTemplate.from_template(
        "You are a museum guide. Reply briefly."),
    MessagesPlaceholder("messages"),
])

def chatbot(state: MessagesState):
    messages = prompt.format_messages(messages=state["messages"])
    return {"messages": [llm.invoke(messages)]}
```

The node sends the instruction and the messages; it returns only the reply.

Without a Checkpointer



Earlier calls are not restored.

Open 01_memory.ipynb: 2. A chatbot node

Checkpointer and Threads [1]

Checkpoint

A snapshot of the graph state, saved after each step.

Thread

One conversation, identified by a thread_id.

Same thread_id: the saved state is restored before the graph runs.

Compile with a Checkpointer [1]

```
from langgraph.checkpoint.memory import InMemorySaver

saver = InMemorySaver()
memory_graph = builder.compile(checkpointer=saver)
lea = {"configurable": {"thread_id": "lea"}}

memory_graph.invoke({"messages": [HumanMessage("My name is Lea.")]}, lea)
memory_graph.invoke({"messages": [HumanMessage("What is my name?")]}, lea)
```

Send only the new message; the earlier ones are restored.

Separate Conversations [1]

thread_id = "lea"

My name is Lea.
What is my name? → Lea

thread_id = "marco"

What is my name? → I'm not sure.

A new thread_id starts an empty conversation.

Open 01_memory.ipynb: 3. Checkpointer and threads

Where Checkpoints Live [1]

InMemorySaver

Kept in the Python process.
Lost when the kernel or server restarts.

Persistent savers

SQLite or Postgres.
Conversations survive restarts.

Same graph code; only the checkpointer changes.

Delete a Saved Conversation [1]

```
saver.delete_thread("lea")
print(memory_graph.get_state(lea).values)

# The next call starts a new conversation.
result = memory_graph.invoke(
    {"messages": [HumanMessage("What is my name?")]}, lea)
```

Delete one thread. Other conversations keep their state.

A Rolling Buffer [3]

```
MAX_MESSAGES = 6

def buffered_chatbot(state: MessagesState):
    recent = state["messages"][-MAX_MESSAGES:]
    reply = llm.invoke(prompt.format_messages(messages=recent))
    return {"messages": [reply]}
```

The state keeps everything; the model sees only the latest messages.

Exercise: Two Visitors

Two threads

Lea likes green, Marco likes blue.
Alternate between their threads.
Ask each colour back.

A short buffer

Set MAX_MESSAGES = 2.
Inspect the recent messages.
Compare with the saved state.

Open 01_memory.ipynb: 4. A rolling buffer + exercise

Next: The Avatar Remembers [1]

Same graph in the server

response(text) invokes the graph with a checkpointer.

Thread in the backend

The browser sends only text.
The server chooses the thread_id.

Chapter 03 connects this memory to the virtual human.

References

[1] LangGraph persistence

Checkpoints, threads and saved state

[2] LangGraph Graph API

Reducers and MessagesState

[3] LangGraph memory

Short-term memory and trimming messages